

Practical Course Report

Creating a Graph Harvester

Jonas Spies

Date of Submission:

Advisors:

Prof. Dr. Alexander Wolff

Tim Hegemann, M. Sc.



Julius-Maximilians-Universität Würzburg
Lehrstuhl für Informatik I
Algorithmen und Komplexität

1 Abstract

In an attempt to answer the question of what makes a graph interesting, House of Graphs have built a database of many interesting graphs. However manual insertion of new graphs being tedious creates the need for an application that automates this process. While such a system already existed, it suffered from only being able to reconstruct roughly 35%¹ of graphs exactly and the necessity of hosting a server due to the project being written in Python. Thus, I have rebuilt a similar application in Typescript to allow client-side execution while deviating from the original workflow in some aspects in the hope of improving the detection rate. Using graph drawings from papers featured in the International Symposium on Graph Drawing and Network Visualization from 1998 to 2024, I estimated the new implementation’s performance against that of the old one, and came to the conclusion that the two systems’ detection rates are similar. The main reason why further improvement may be difficult is the trade-off that arises with the adjustment of some parameters: if they are changed to properly detect one instance, another instance may not be able to be detected correctly.

2 Introduction

Graph theory is a major field in mathematics and computer science due to a graph’s ability to represent objects and their relations to each other in a compact and intuitive manner. This representation allows all kinds of problems to be solved in a unified approach, such as the problem of finding the shortest path from one location to another, as often applied in navigational devices.

Many of these graphs may have certain properties that make them inherently more interesting than others. For example, they may serve as a particularly illustrative example to show an algorithm’s best or worst case performance, as a counterexample to a plausible conjecture, or as an extreme instance where some property, for example planarity, still holds. While it remains difficult to define what makes a graph “interesting”, we use the appearance of a graph in a scientific paper as a major indicator for potential interest, leading our focus on to graphs featured in them.

In an attempt to give the word “interesting” a more concrete meaning, House of Graphs (HoG) have made it their mission to create a database that contains many such graphs. However, scanning through hundreds of papers manually is not only prone to errors especially for large graphs, but also very time consuming, which is why we aimed to write a program that can automate this process.

Thus, a graph harvester app has been developed in Python to automatically scan scientific papers and heuristically extract any graph contained in them in an SVG format, returning a visualization, a Graph6 String, and a link to the respective database entry or to the upload form for that graph. A user can then use this tool to upload new graphs with the click of a button.

¹Only counting SVG drawings. Manual evaluation found 419 true positives, 248 of which were graded ‘A’, and 287 false negatives.

I have rebuilt the project in Typescript to allow client-side execution of the code, thus eliminating the need of hosting a server, while achieving similar performance compared to that of the previous system – reusing the previous frontend built with React for visualization. While following the general work pipeline of the previous implementation, I allowed myself to experiment with the concrete algorithms used in the background to try and make the application as efficient as possible while also adding and omitting a selected number of processing steps. For example, I widened the definition of a vertex from “a rectangle or a circle” to “any object of limited complexity that forms a closed loop”. This change, however also required a more intricate detection of arrow heads.

Using papers featured in the International Symposium on Graph Drawing and Network Visualization from 1998 to 2024 to measure progress during development, I have also made some observations about the general structure of these papers and their respective graph drawings, as well as on what makes a graph drawing “difficult” to scan heuristically. Furthermore, I will highlight several trade-offs one has to consider when adjusting the system’s parameters.

These observations could serve as a starting point for anybody aiming to improve the project, or as a guideline on how to make machine-readable graph drawings for researchers who wish to find their graphs included in the HoG database.

3 System Limitations

In order to scan graphs, we limit ourselves to vector graphics because those consist of multiple drawn objects that can be examined heuristically to discern between vertices and edges. While the ability to scan bitmaps as well would be promising, this would require additional effort such as working with convolutional neural networks to extract patterns. With only roughly 17% of all graph drawings being drawn as a bitmap, we deemed the extra effort not worth it, although future work may easily improve the performance by supporting bitmaps as well.

Besides that, we limit ourselves to undirected, simple graphs, so that any two vertices u, v with an edge from u to v also contains an edge from v to u . This information is stored in a single edge (u, v) (or alternatively (v, u)), and each pair of vertices can only have up to one edge in the graph. While there may also be many interesting directed graphs or multigraphs, House of Graphs does not support them, thus we do not discern between them.

Lastly, there are many graph drawings that only include four or less vertices. While one could argue that such graphs are interesting as well, the likelihood of that instance already being on the database is high, which is why we limit ourselves to graphs that have at least five vertices, four edges, and no isolated vertices (isolated vertices will just not be considered part of the graph during detection).

4 The old implementation

4.1 Workflow

The old implementation handles one document at a time, going through that document page by page.

Identifying figures As a first step on each page, the old pipeline groups drawn objects by figure, so that a matching between figure and extracted graph can be established while further narrowing the number of objects to be considered at a given time. This is achieved by a simple algorithm where each drawn object on a page starts off as its own figure. If two figures have overlapping bounding boxes, they are merged to a single figure containing the drawn objects of both. By repetition until convergence, this approach works surprisingly well to correctly identify each figure as such, which usually even gets reassigned its original figure number through another simple search. This speeds up manual evaluation because each graph produced by the harvester can that way be traced back to a unique figure in the paper.

Initial vertex and edge candidates Iterating figure by figure, the system tries to construct circles from bezier curves, so that initially, all rectangles and circles are a vertex candidate and remaining objects are edge candidates. An initial filter clusters the vertices by size, then ensures for larger than average vertices that they do not contain any already accepted vertices and that they have some edge candidate starting inside. Any vertices that do not pass this filter will from this point on be treated as a group of potential edges. After that, duplicate vertices are thrown away completely, which is important because any drawn objects with both a fill color and an outline will appear twice.

Graph detection From there, it discerns between edges as straight line segments and edges as bezier curves, and looks for any edge candidates starting or ending in a vertex candidate. If both the start and endpoint of a segment are incident to a vertex, that segment is an *edge*. If only one endpoint is incident to a vertex, it is called a *half-orphan*. Remaining segments are *orphans*. Orphans and half-orphans are then extended in a direction to see if a vertex can be found that way. With orphans and half-orphans that still remain after this being thrown away, the system checks if any vertices lie between two endpoints and splits each such segment accordingly.

Lastly, it performs a depth-first-search to find all connected components, and returns each component as one graph, which is part of an object that corresponds to the figure.

4.2 Limitations

The old pipeline performed poorly on instances that have unconventional shapes as vertices, very large vertices, or very bright objects. Instances with an underlying grid also proved troublesome, since it is difficult to discern a grid from actual edges, as

some instances do feature grid drawings. Also, graph drawings with no explicit vertices, i.e. drawings where the vertices are only implied at positions where edges meet or bend, naturally cause the system to fail.

5 Personal Contributions

5.1 Goals

The goal is a graph harvester that runs client-side in a browser, so that the user can upload PDF files and expect to reliably be shown a figure in the paper, as well as the corresponding graph. The user can then decide if he wants to edit the graph using HoG's built-in editor, or if they want to upload the graph to the database. Because the user is ultimately in control, it is best to have as few false negatives as possible, even if that means having more false positives.

5.2 Benchmarks

Besides the web application, I have built a second entryway to the pipeline which runs through two hardcoded directories, one containing a set of reference graphs, and one containing a set of scientific papers in PDF format. The system will then iterate the pipeline over all papers and in each step try to match as many of its graphs to an equivalent reference graph belonging to that paper. While the equivalence check is deliberately simple, only comparing the number of edges and vertices, I have found it sufficient for the purposes of this benchmark due to the inherently small number of graphs featured in a single file. This allowed me to focus on actually improving the system rather than implementing a complicated exponential runtime algorithm for graph isomorphism. The reference graphs come from the old pipeline, but were manually checked to only contain graphs graded 'A', a rating given when the extracted graph is isomorphically identical to the depicted graph, i.e. an instance where the previous system succeeded without any errors. This means that any time 'recall' is mentioned, it is by no means a measure against ground truth, but more of a measure of agreement with the old implementation, given instances the old implementation performed well at. While I did put in some effort to get closer to ground truth by manually constructing further reference graphs that the previous implementation failed to detect correctly, I quickly decided that this would not be worth the effort. Even a single small instance could easily take me 15 minutes, with some graphs that contain hundreds of vertices practically impossible to accurately reconstruct by hand. The process of fixing a given graph made by the old pipeline starts by looking up the graph's grade to ensure it is not part of the data set yet, then opening the paper and looking up the respective figure that the graph originates from. While the figure number was correct most of the time, there were some errors that caused confusion. Comparing the adjacency list the old pipeline constructed to its respective figure required tasks such as figuring out which vertex in the picture is v_1 , v_2 , etc, which can be tedious and prone to errors. After that, checking which vertices and edges were missing or needed to be removed, required a high level

of concentration as to not make any mistakes along the way. As a result, I decided to throw away the few extra instances I created and instead focused only on the graphs originally graded as ‘A’. With only 248 out of the total 706 graphs in the data set being graded ‘A’, there is a lot of potential improvement my benchmark is unable to show, which means that generally, my benchmark underestimates the performance of the new implementation. For example, a graph harvester that could accurately determine all the 458 graph drawings the old application failed at, but none of the 248 graph drawings the old application succeeded at, would still achieve 0% recall in this benchmark. With that being said, the most recent benchmark over all papers that included at least one graph graded ‘A’ was able to match 222 out of 248 graphs for a recall of 89.5% and found a total of 578 graphs for a precision of 38%. Note that, even though precision is fairly low, the actual precision against ground truth is likely higher than what this benchmark can indicate due to the same reasons discussed before. I have additionally manually found some instances where my implementation performs better than the old one – for example GD10_383-388 Fig. 4 or GD14_264-276 Fig. 3, which leads to my conclusion: Even though the new implementation fails at some instances the old implementation succeeded at, it also succeeds at some instances the old implementation failed at, so that the two systems seem overall similar in performance.

5.3 The general workflow and impact of different implementations

Similar to the old graph harvester, I use MuPDF to extract vector graphics page by page, then group them together into figures by overlapping bounding boxes. From there, I use various heuristics to interpret these vector drawings as a graph drawing.

5.3.1 Initial vertex and edge selection

The first difference to the old system arises at the initial selection of vertex and edge candidates: I iterate over every drawn object and simply check if it forms a closed loop. With a lot of papers drawing letters as a vector graphic, this produced many instances where letters would be interpreted as vertices, which is why I eventually also added a complexity filter, limiting how many bezier curves are allowed in a vertex. Additionally, many drawings feature directed edges or arrows to highlight certain vertices. The arrow heads would then be falsely interpreted as a vertex, so that I built a simple arrow head detection that would mark any object with a sharp angle of 45° or less as an arrow and thus reject it as a vertex. Any object not selected as a vertex would then be broken down into its separate strokes, interpreting each of these strokes as an edge candidate. Rejected arrows are currently reused as edge candidates because the current detection is too broad, so that throwing them away would cause problems for many graph drawings. A potential solution for this is briefly discussed in Section 6.

5.3.2 Additional filters

While the selection of vertex candidates was deliberately lax initially, it became clear that further filtering the selection of vertex candidates was key to improving recall: an

earlier version without any of the following filters only found 30 ‘A’ graded graphs, as opposed to the current version which found 222. One filter that misses from the old model, however, is that of checking for duplicate vertices. While there are many other filters in place that handle duplicate vertices, the absence of this filter is actually a prime candidate for the explanation as to why sometimes vertices seemingly blow up in size compared to the original drawing, as discussed in Section 6

Merging overlapping vertices Due to the nature of how vector graphics are drawn, most vertices actually produce two objects: one for the outline and one for the interior color. If both survive the initial selection process, it becomes essentially random which of the two objects is selected to become linked to an edge, due to an edge only being able to store one vertex per endpoint. This would cause problems during the last step discussed in Section 5.3.4, so that many vertices would be split into two and some components of a graph may end up missing entirely. As a result, an initial filter which merges any two vertices into a large single one, if they overlap by a certain threshold, became the single most important feature to be implemented, boosting the detection rate from an initial 30 instances, all the way up to 173 instances found. However, the following filter would largely take over this role, so that actually merging vertices started becoming more of a backup system in my implementation. Still, instances like GD13_299-310 Fig. 8, where there are two offset circles depicting the same vertex, prove that a feature like this remains necessary.

Clustering vertices by area Largely copied from the old graph harvester, this filter first clusters vertices by the size of the area they take up, then splits each cluster into two groups: the accepted group, and the group up for further examination. If a cluster contains too few elements, or if its mean area is larger than the average over the remaining clusters, it would be sent to further examination, while other clusters would be accepted. A cluster up for further examination then needs to ensure that it does not contain any center point from the set of already accepted vertices. This check largely ensures that if there is one object for the outline and one for the interior of a vertex, just one, usually the interior due to its smaller area, is chosen. However, the parameter of a minimum cluster size interestingly enough turned out to be a hugely influential parameter: While I eventually got it down to a value of 3, pushing it down further comes at a huge cost of detection rate, likely due to less actual vertices being accepted without any further examination. This creates the opportunity for huge objects to get accepted as a vertex, if they are examined before any of the actual vertices that it overlaps. That way, the huge vertex would remain accepted, and later merge with all actual vertices into one big vertex, swallowing parts of the actual graph drawing. While I believe that there could be better approaches to balance an initial selection of vertices, this one proved good enough, so that I found it hard to manually find an instance where this approach causes problems.

Height-to-Width Ratio Another filter that seemed promising at first was that of a filter based on the height to width ratio of a vertex. I eventually optimized the parameter so that any vertex that has less than 75% of its width in height, or of its height in width, would be rejected. While this initially gave a small boost in recall, recent tests have shown no difference. However, there was one instance, GD98_369-376 Fig. 6, which could only be matched without this filter, and one instance, the figure on page 70 on GD17_84-97, which could only be matched with this filter active. A further optimization could be to only use the filter, if some vertices still remain after, but in its current form, the impact of the filter seems to be weak.

Size of vertex compared to total size of figure While simply filtering vertices if their area is too large relative to that of the figure seems like a good idea, in practice other filters already seem to do this job. As a result, no difference in recall could be observed when changing the threshold for this variable within reasonable bounds. For example, filtering any vertex that takes up 20% or more of the total area, compared to already filtering it if it takes up 5% of the total area, does not change recall at all, and setting the threshold too low naturally hurts recall, rather than helping it. However, one interesting observation is that contrary to intuition, the total number of detected graphs is higher with a stricter threshold: having detected a total of 611 graphs with a threshold of 5% as opposed to just 578 graphs detected with 20%. This could indicate that there are some niche instances the old pipeline could not properly detect, that do rely on this filter, which motivated me to keep the threshold relatively low at 5%.

Brightness Filtering objects, not just vertices, by brightness could be a powerful tool to get rid of many objects that make an instance difficult to detect, such as grids and background colors. For instance, GD15_243-256 Fig. 1 has a light gray grid laying under one of its graph drawings. Creating a brightness scale from 0 (black) to 1 (white) and completely throwing away any object over a certain threshold can make many instances easier. For example, a threshold of 0.87 would cause the aforementioned graph to lose its underlying grid. However, overall 0.87 seems like it is already too strict, as many instances such as GD20_56-70 Fig. 5 break at this point as well. In total, a threshold of 1 compared to one of 0.87 detects 222 as opposed to just 215 graphs from the data set, which unfortunately indicates that, while there is a trade-off of sacrificing some graphs to be able to detect others, it is generally not worth taking currently since it means losing graphs in total.

Arrows While the old implementation with its strict definition of a vertex had no problem with arrow heads, my new implementation does suffer from detecting any arrow head as a vertex, leading to many falsely detected instances. To fix that, I implemented a simple arrow head detection, that would reject any object as vertex candidate, if it contains a sharp angle, namely 45° or less. As a small optimization, the system actually does not compute the internal angles, but instead the cosines, where a cosine of 0.70710678118 or larger corresponds to such an angle. In order to compute the internal

angles of an object, I simply do pairwise comparisons between any two strokes that meet in a common point. With the first iteration of this algorithm simply walking along the outline of an object to find these pairs, I quickly encountered a bug that caused many arrow heads to not be detected. The fix lied in the interesting observation, that some objects are actually drawn in a way where they close the loop through walking along the perimeter, and then close the loop again by connecting the first and last endpoint of the object's path. Since those are already identical, this resulted in many arrow heads having one line segment of length 0, so that if two adjacent line segments form a tight angle, that pair would never be checked because along its perimeter, a line segment of length 0 lies between the two segments. By fixing that bug, arrow head detection allowed correct detection of twelve additional instances, so instead of 210 graphs, 222 could be found.

5.3.3 Implied Vertices

A large difference to the old pipeline is an attempt to detect implied vertices. To do that, I simply check for an instance if there are any vertex candidates left after initial filtering. If not, the system will look for any point where three or more meet in an endpoint, and create a vertex incident to these edges at that position. Eventually I tried creating implied vertices at a point even if just two edges meet there, but this caused hundreds of false positives due to any drawn object with at least six segments, such as letters, suddenly turning up as a detected graph, which led me to revert this decision. However, none of the graphs with implied vertices are represented in the benchmark due to the old implementation not having this feature. Determining the actual impact of the feature on the detection rate thus necessitates further manual evaluation.

5.3.4 Creating an incidence matrix

With the set of vertices being determined, the next step would be checking which edges are incident to that vertex. Although most vertices lie on an endpoint of an edge segment, there are many instances where vertices lie between two endpoints, which led me to the following algorithm: First, divide the segment into many sample points (My results come from using 30 points), with the first and last being the two endpoints and the others being equally spaced along the segment. Secondly, check for each vertex if any sample points lie within its bounding box, scaled by some lenience factor (best results with 1.07). The concrete implementation of the algorithm was RBush². The result is an incidence list of edge segments for every vertex, that will then be used to split every segment so that no segment has a vertex lying between its two endpoints, and every edge segment knows if it is incident to a vertex at each of its endpoints. With an edge being a path from one vertex to another, there may be instances where multiple vertices are incident to a single endpoint. In such a case, I take the vertex minimizing the distance to the endpoint, which is a solution that seemingly has not caused any problems.

²<https://github.com/mourner/rbush>

After that, I use KDBush³ to find all orphaned edge endpoints incident to another orphaned edge endpoint, i.e. a pair of incident endpoints that are both not incident to any vertex. While the search radius has caused some troubles since there are both instances where two strokes form one edge, yet they don't exactly meet in a given point, and instances where two separate edges come very close to each other at a given endpoint, the best results could be achieved with the search radius being either the stroke width, or 0.25px, depending on which is larger, with very short segments not getting the guarantee of at least 0.25px. This allows chaining different segments together into one large edge, which is important for many instances: in an earlier version where this was not possible, only 22 out of the 248 'A' graded graphs could be matched. After implementing the chaining of edge segments, the recall jumped up by 36%, up to a total of 30 matched instances.

Lastly, remaining half orphans are rescued by linearly extending them into the direction they point at. For bezier curves, the extension goes into the direction obtained by shooting from the last control point towards the orphaned endpoint. Since this implementation also caused many false positives, I had to carefully limit the extension, up to a maximum of nine pixels, or seven percent of the edge segment's length.

The resulting incidence list can then be turned into a graph using depth-first-search, just like the old graph harvester did.

6 Limitations and possible solutions

Arrow Head Detection The system currently relies on reusing arrow heads as edge segments. This causes many perfectly fine edges to attach the arrow's stroke segments to its end, rather than the vertex it is incident to. While that does not cause any problems in itself, it causes the last segment of the edge to be very short, thus extending that segment by seven percent achieves much less, than it could achieve if arrows were not reused as edge segments. However, in order to enable that, it would be interesting to analyze cases where an arrow is actually important for edge detection. Possible indicators could be size of the arrow, and number of vertices it is incident to. For example, if the arrow consists of three line segments, there could be three vertices the arrow is incident to. If there are more or less vertices in its vicinity, the arrow might be safe to discard as an edge.

Large vertices The implementation often bloats vertices up to be much larger than they appear to be in the original drawing, causing problems by finding additional incident edges and falsely merging two bloated vertices into one. It remains to be seen if this is due to a bug, or some other phenomenon that can be counteracted. A plausible explanation could be, that some papers might draw their vertices with a blown up interior that they then crop by the outline of the vertice. Since the cropping outline is another object, that blown up interior would then stay as large, and might be able to sneak through the filters designed to catch large vertices.

³<https://github.com/mourner/kdbush>

Grids It remains difficult to discern grids lying under a graph to grid graphs. While a filter based on brightness or stroke style could help, there are still many instances that would break by taking these factors into consideration. For example, currently any drawn object with a luminance value of 0.95 is discarded, with 1 indicating a fully white object. Some instances, such as GD15_243-256 Fig. 1 have a grid that can be filtered by setting the threshold to 0.87, but that would, at the same time, falsely throw away a vertex displayed in GD20_56-70 Fig. 5.

Edge Extension Many instances, such as GD09_32-43 Fig. 2 feature edges that end far away from their incident vertices, yet increasing the extension threshold from 9px to a higher value would break many instances in return, such as GD09_303-314 Fig. 1a). This seems to be a problem especially for short edge segments. A possible reason could be because in its current form, if a vertex has both an outline and an interior color, and one of them (usually the outline) is rejected, the strokes that make up that rejected object are still reconsidered as an edge candidate. If, by coincidence, an edge does not connect to the vertex directly, and instead connects to one such re-used outline strokes, there may be a situation where such a reused outline stroke ends up as a half-orphan. By extending the half-orphan in what is essentially a random direction, a compact graph drawing may falsely connect this half-orphan to another vertex, ruining the detection for that instance, even if this phenomenon occurs just once. Another interesting observation would thus be to include a filter especially made for vertices that are represented by two objects, and see if this allows the edge extension threshold to be chosen a bit more generously.

Forks Some graph drawings, such as GD17_521-534 Figure 1d), feature edges with a long shaft, that split into multiple thinner edges to connect multiple vertices at once. With the current implementation of the graph harvester, this is not possible to detect correctly due to the following reason: Edges are hardcoded to be a path, not a tree. That means that at the endpoint of an edge segment, there can only be a link to up to one object, not multiple as is the case with such forks

This last quote is to make the compiler happy[AFK⁺25].

References

- [AFK⁺25] Hugo A. Akitaya, Sándor P. Fekete, Peter Kramer, Saba Molaei, Christian Rieck, Frederick Stock, and Tobias Wallner: Sliding squares in parallel, 2025.
<https://arxiv.org/abs/2412.05523>.